1、MoveIt配置

1、MoveIt配置
- 1.1、启动程序
- 1.2、配置流程
  - 1.2.1、加载URDF模型
  - 1.2.2、创建碰撞免检矩阵
  - 1.2.3、创建运动规划组
    - 1.2.3.1、添加机械臂规划组
    - 1.2.3.2、添加夹爪规划组
  - 1.2.4、创建预设位姿 (Robot Poses)
  - 1.2.5、设置末端影响关节
  - 1.2.6、建立ROS控制器
  - 1.2.7、添加作者信息，不添加，则无法生成
  - 1.2.8、生成配置文件
- 1.3、MoveIt 配置包详解
  - 1.3.1、config文件夹
  - 1.3.2、launch文件夹
- 1.4、配置验证

1.1、启动程序

启动roscore

```
roscore
```

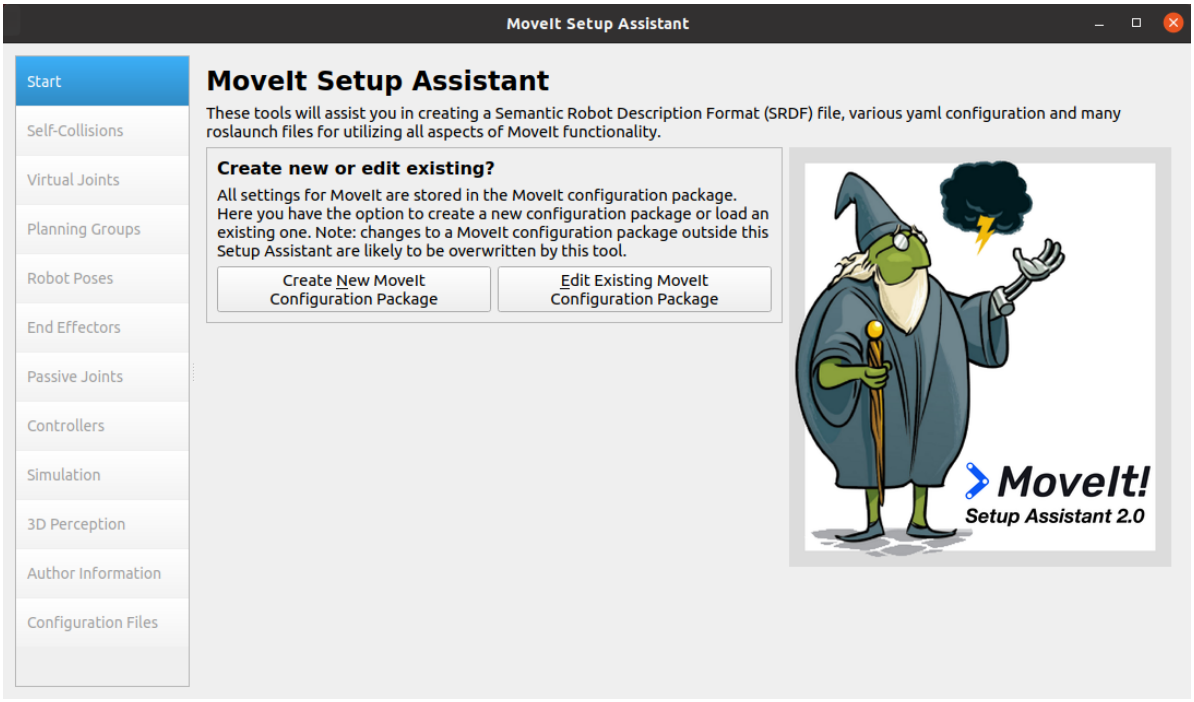
启动MoveIT

```
rosrun moveit_setup_assistant moveit_setup_assistant
```

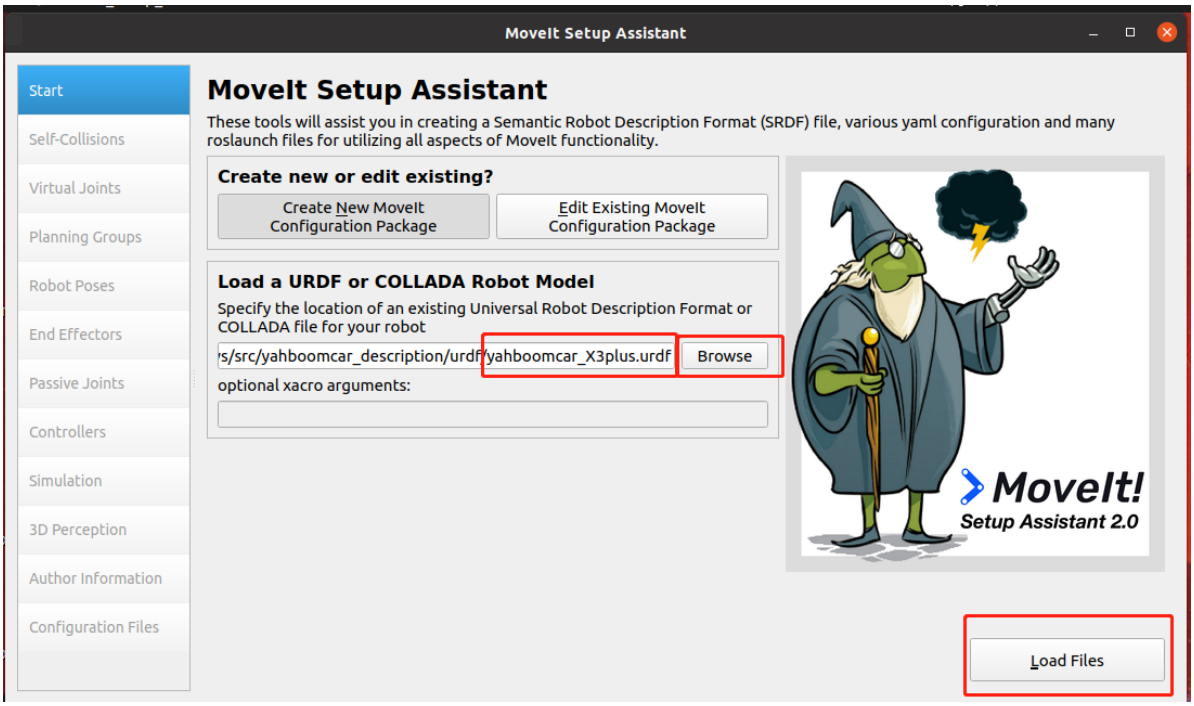
1.2、配置流程

1.2.1、加载URDF模型

如果加载模型时，报错【找不到模型】，退出MoveIt-->进入工作空间-->编译（sudo apt update）-->更新环境（source devel/setup.bash）-->再次启动MoveIt配置。启动后的界面如下：



若是第一次加载模型生成配置，则点击【Create New MoveIt Configuration Package】；若是修改已生成的配置文件，则点击【Edit Existing MoveIt Configuration Package】。

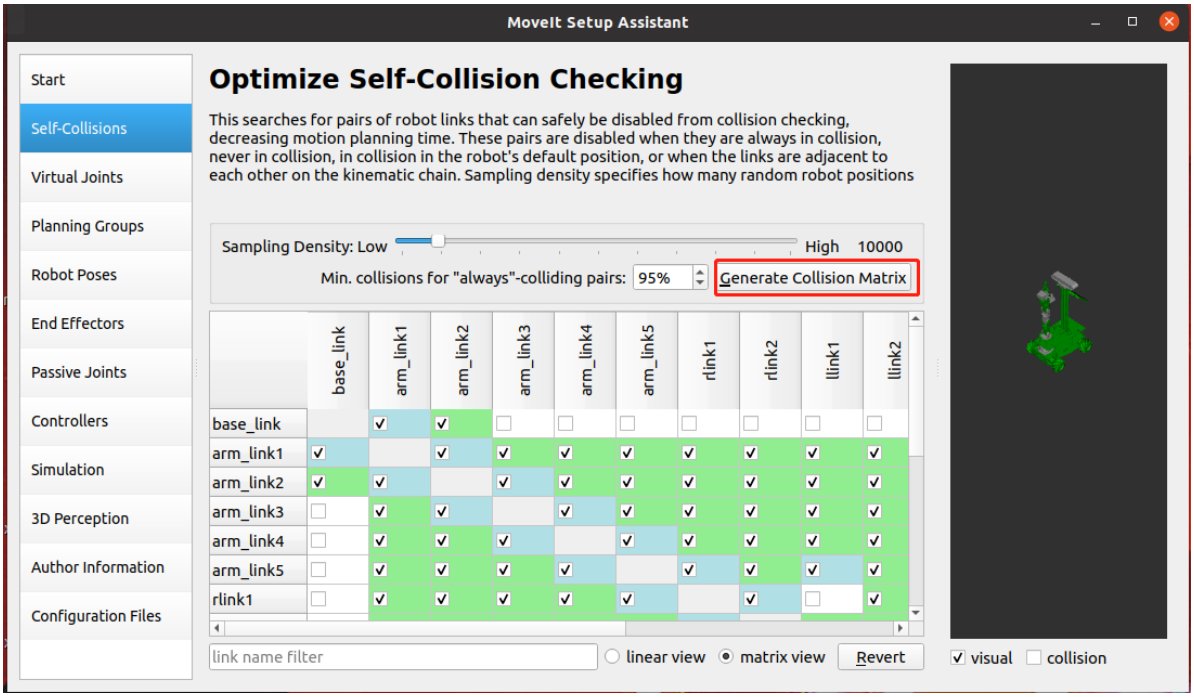


点击【Browse】按钮，找到URDF模型文件，点击右下角【Load Files】加载。

1.2.2、创建碰撞免检矩阵

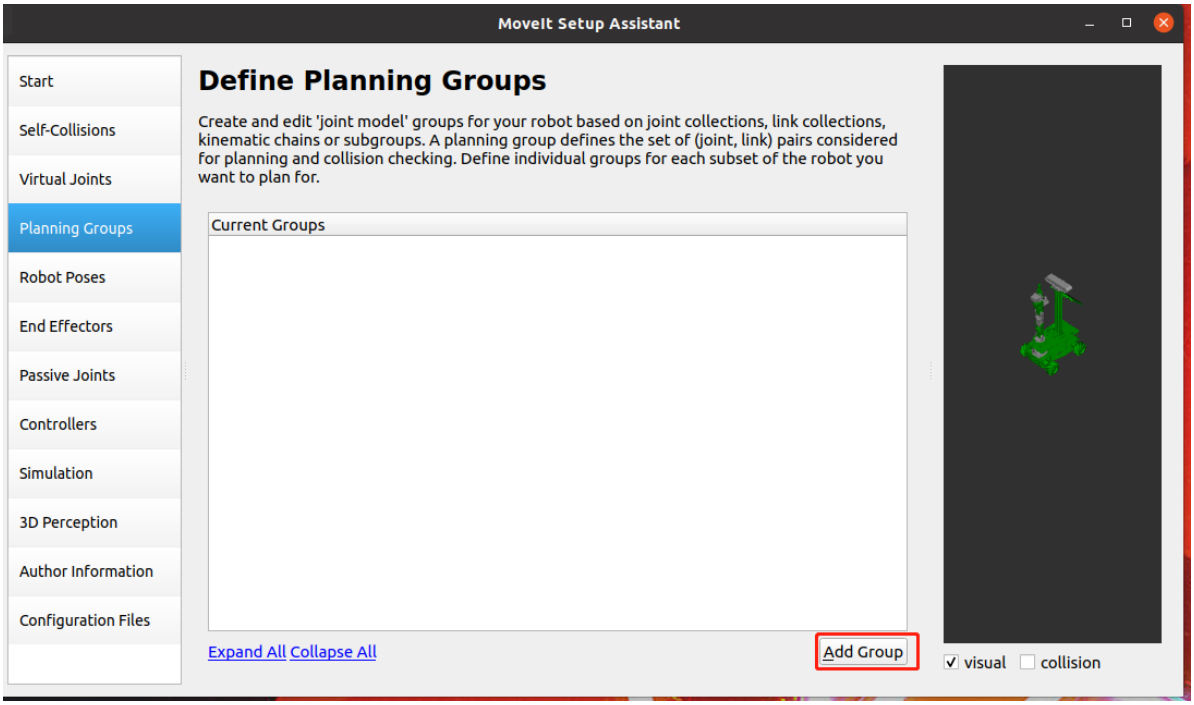
碰撞检测是非常复杂的运算过程。对于多关节机械臂或者类人机器人来说，机械结构复杂，关节多，碰撞检测需要涉及很多的空间几何计算。但是对于刚体机器人来说，有些关节之间是不可能发生碰撞的，比如原本就相邻的肢体。这里生成自我碰撞的目的就是告诉我们，哪些关节之间是不会发生碰撞的。在之后的碰撞检测算法中，就可以直接略过对这些关节之间的检测，以提高检测效率。

模型加载完，选择【Self-Collisions】，点击【Generate Collision Matrix】按钮即可。



1.2.3、创建运动规划组

Planning Group是MoveIt的核心之一。点击【Add Group】，添加规划组



1.2.3.1、添加机械臂规划组

- Group Name：创建组名，设置为【arm_group】。
- Kinematic Solver: 这里我们选择【kdl】

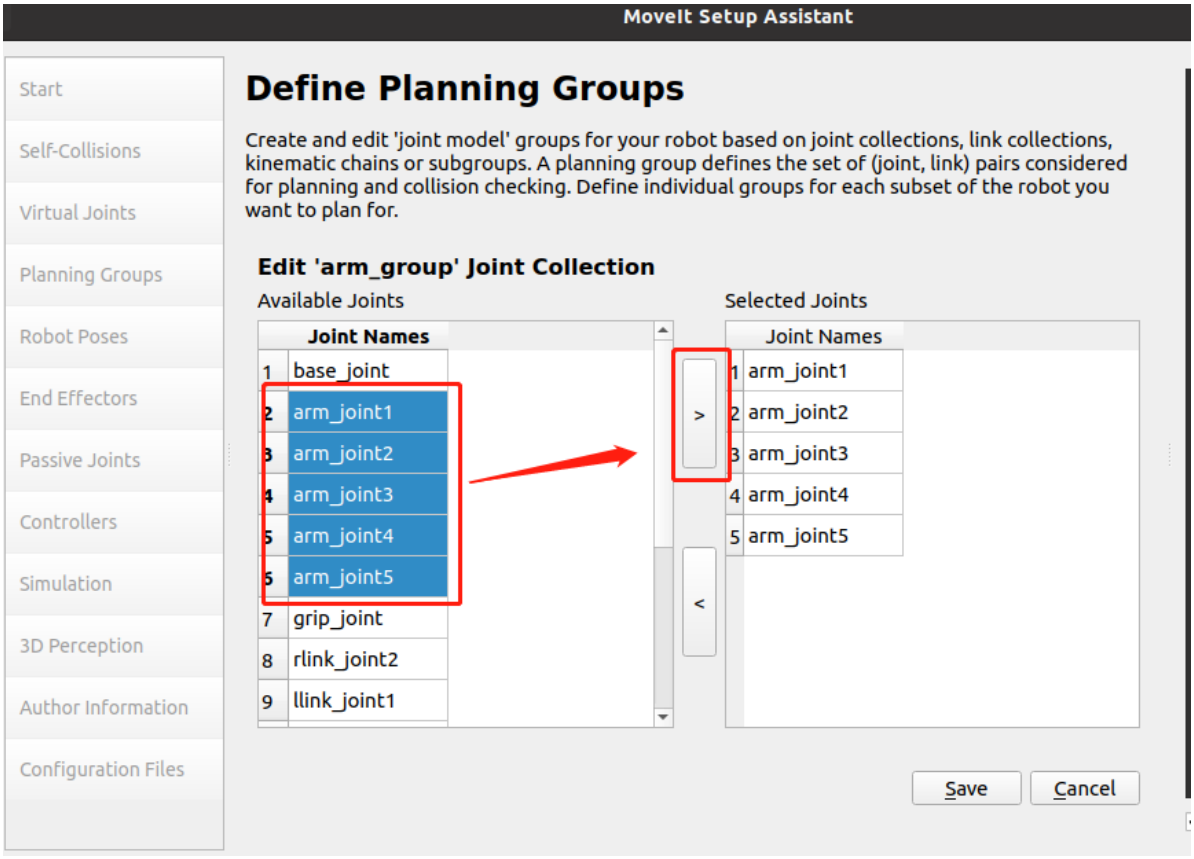
  运动学求解工具，这个就是负责求解正运动学(Forward Kinematics)和逆运动学(IK)的。 一般我们选用KDL， The Kinematics and Dynamics Library。这是一个运动学与动力学的库，可以很好的解决6自由度以上的单链机械结构的正逆运动学问题。

  当然你也可以用其他IK Solver， 比如SRV或者IK_FAST，甚至你可以自己开发新的Solver然后插入进来。

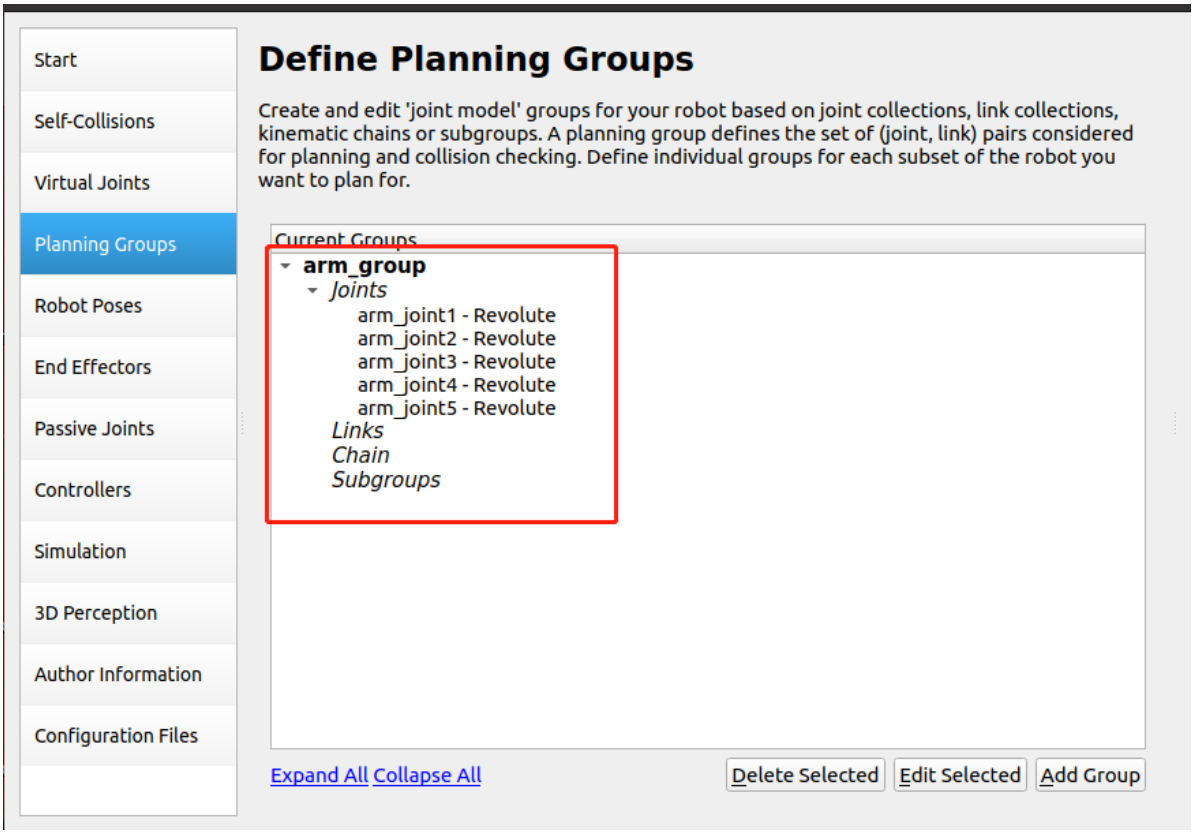
- Kin. Search Resolution: 关节空间的采样密度

- Kin. Search TImeout: 求解时间，如果设备性能不足或者在实际应用过程中规定时间内无解，可增加时间；例如设置为【0.1】、【0.01】。
- Group Default Planner：可以选择RRT，也可以选择None。
- 点击【Add Joints】添加关节。

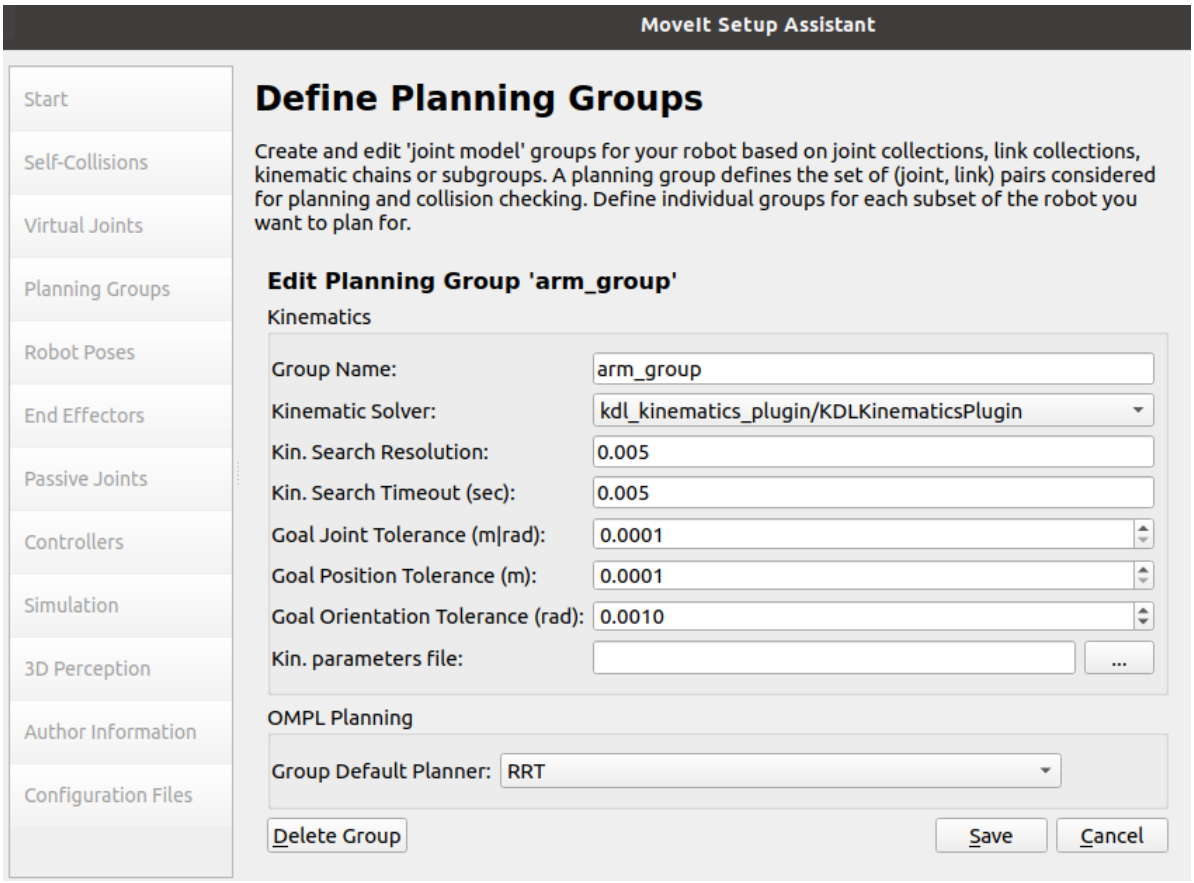
选择arm_joint1到arm_joint5，然后点击>，如下图所示，



点击save保存，完成添加机械臂组，如下图所示，



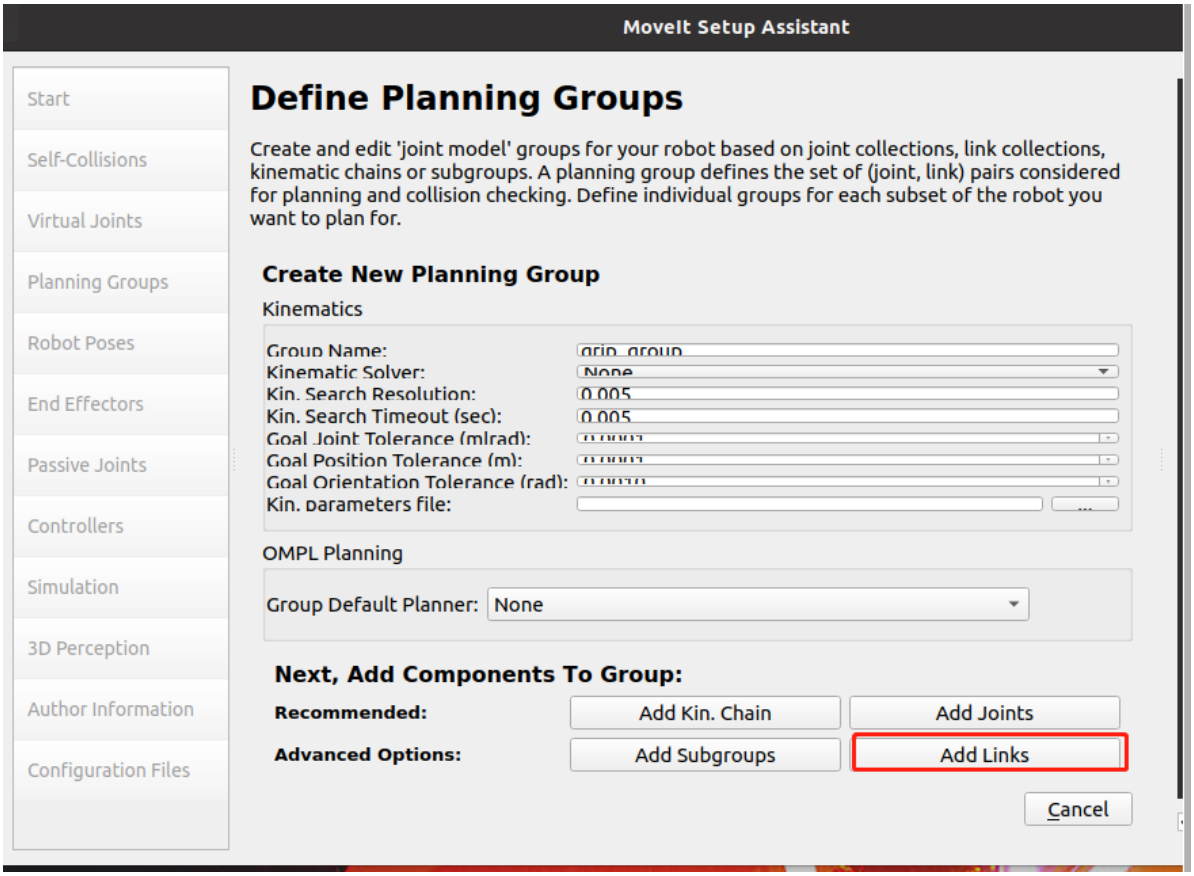
双击arm_group可以查看，



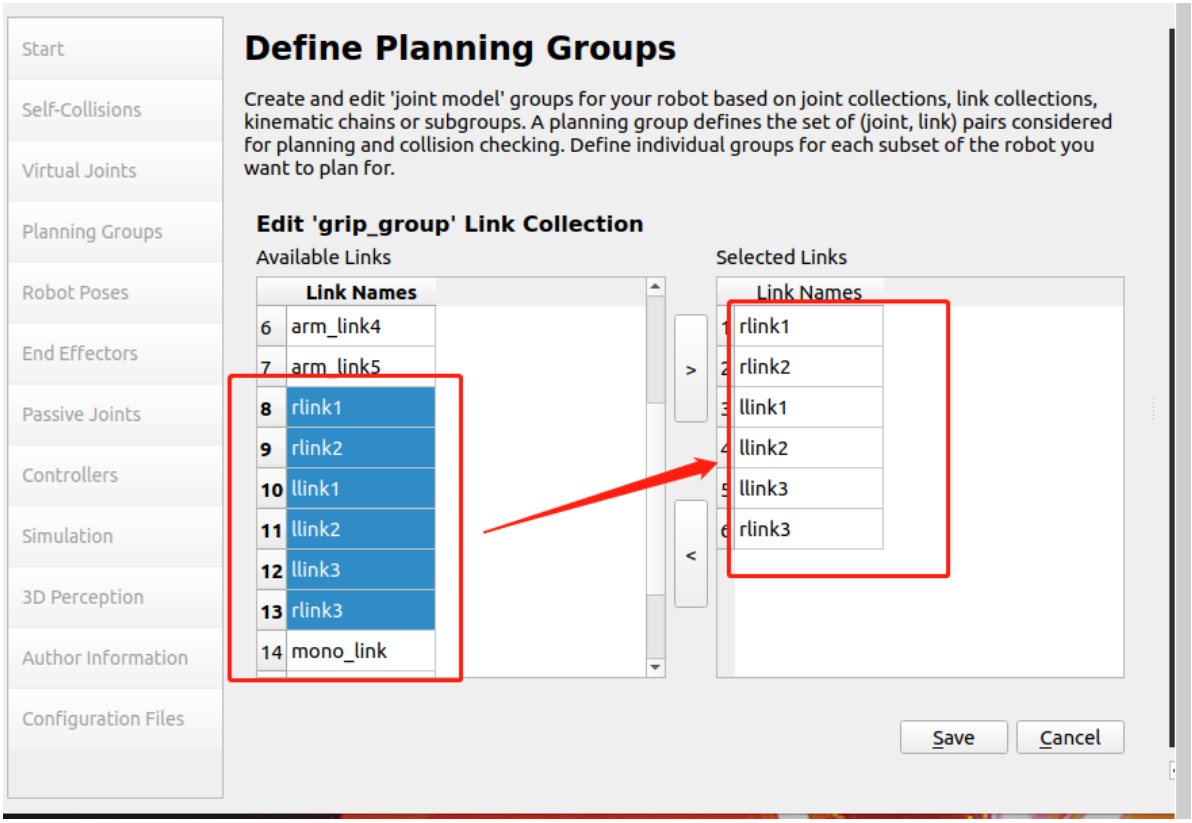
1.2.3.2、添加夹爪规划组

点击【Add Group】，添加夹爪规划组。

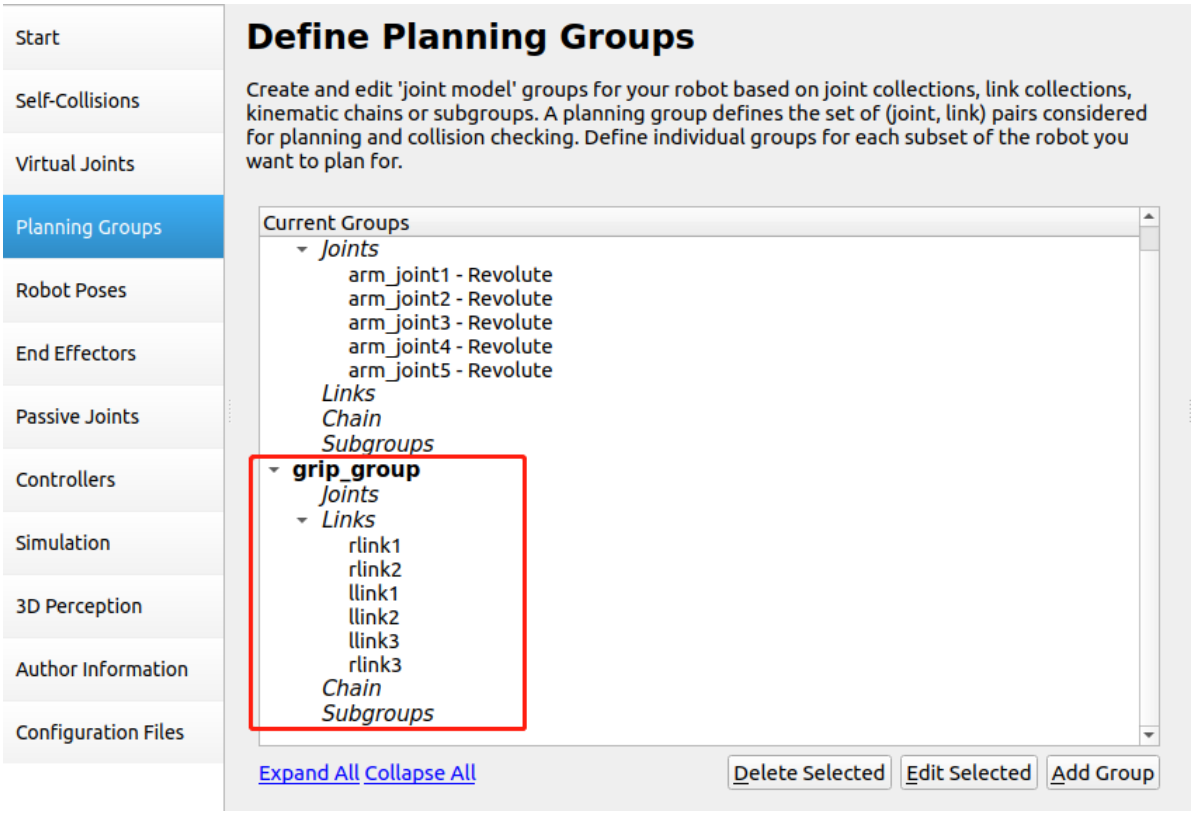
创建组名设置为【grip_group】，无需设置运动学求解器；点击【Add Links】添加夹爪连杆。



选中夹爪部分的连杆，点击【>】，自动添加右侧，点击【Save】保存。



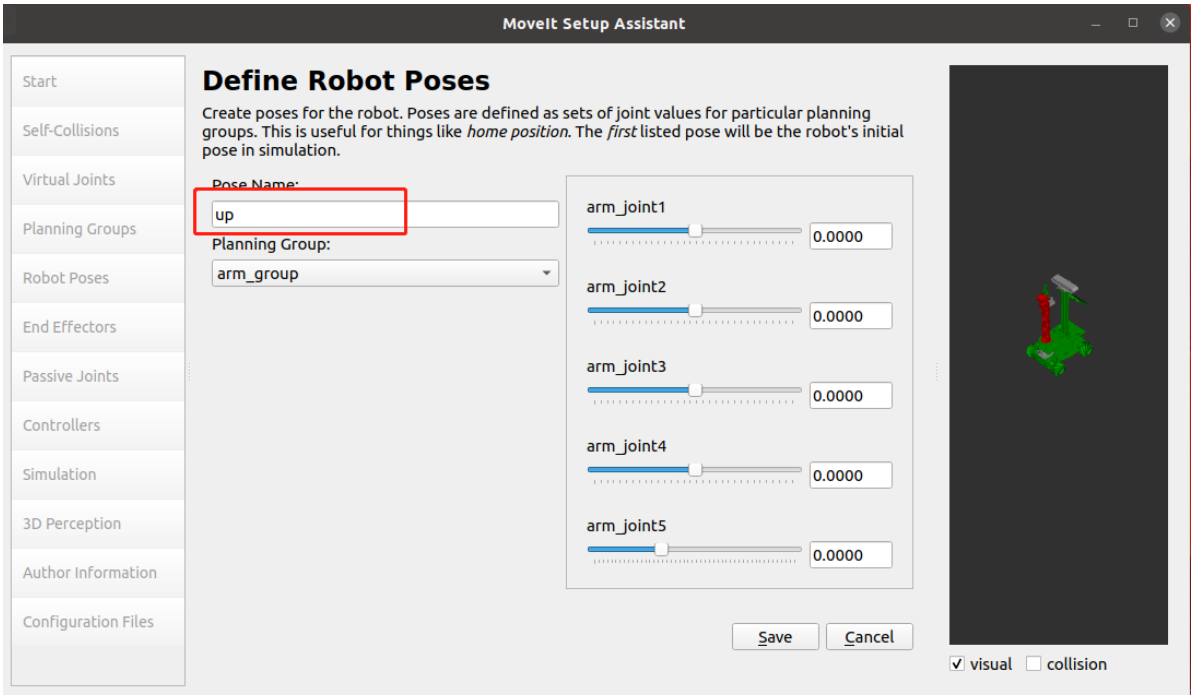
设置好的如下图所示，



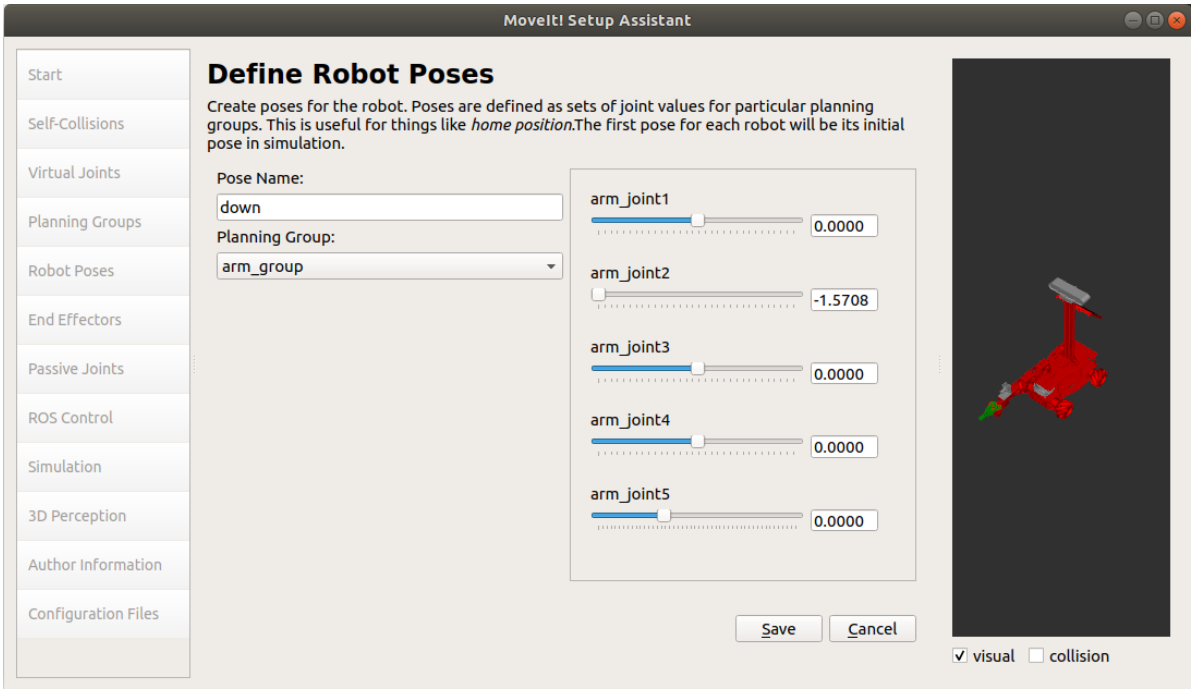
1.2.4、创建预设位姿 (Robot Poses)

选择【Robot Poses】后点击【Add pose】添加姿态。

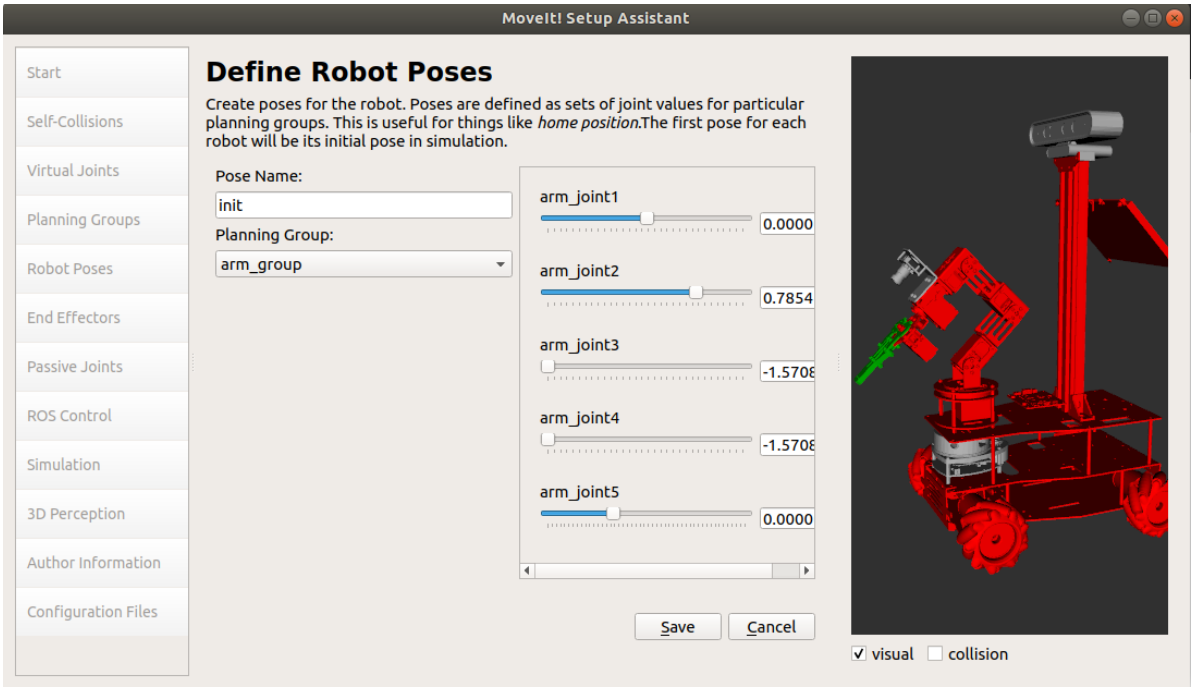
添加机械臂姿态，姿态名Pose Name设置为【up】



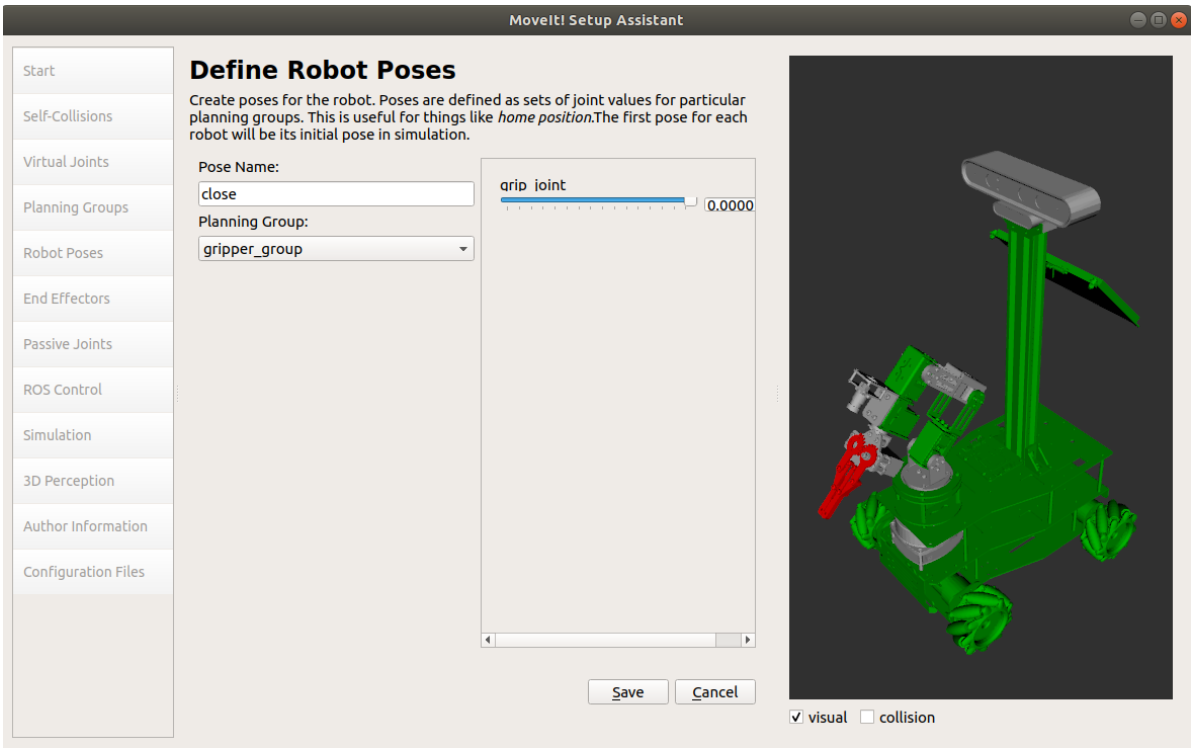
添加【down】姿态



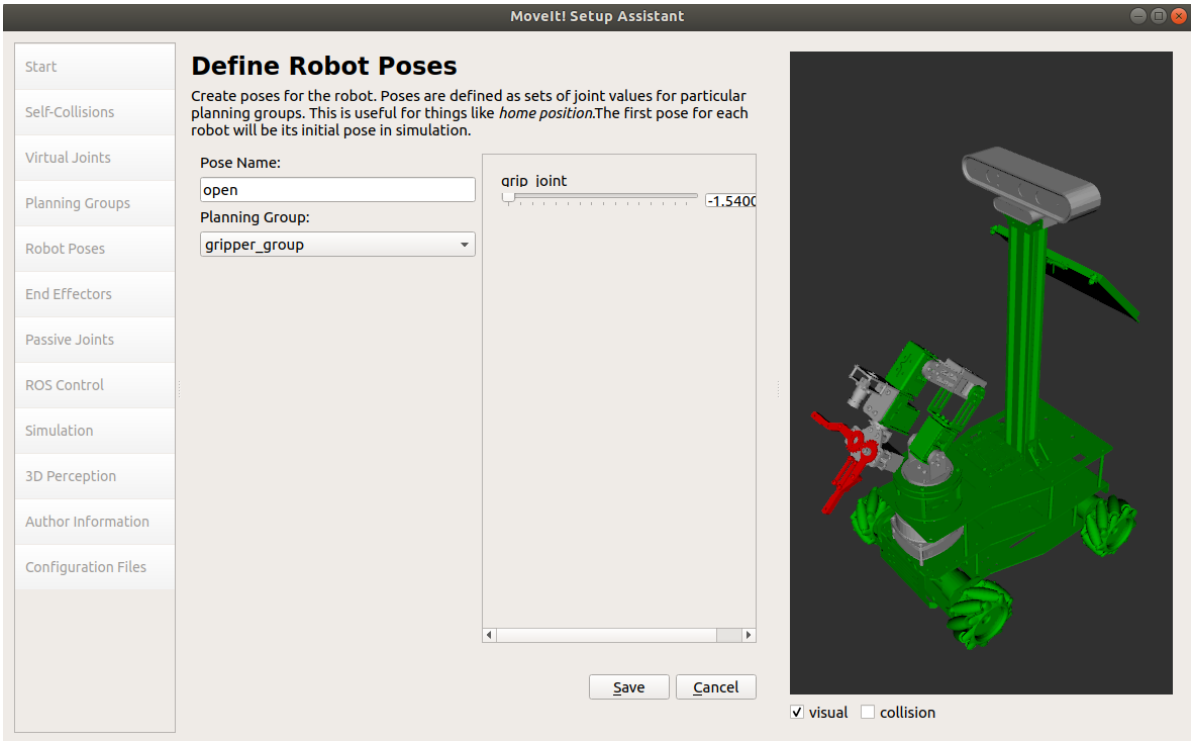
添加【init】姿态



添加夹爪【close】姿态

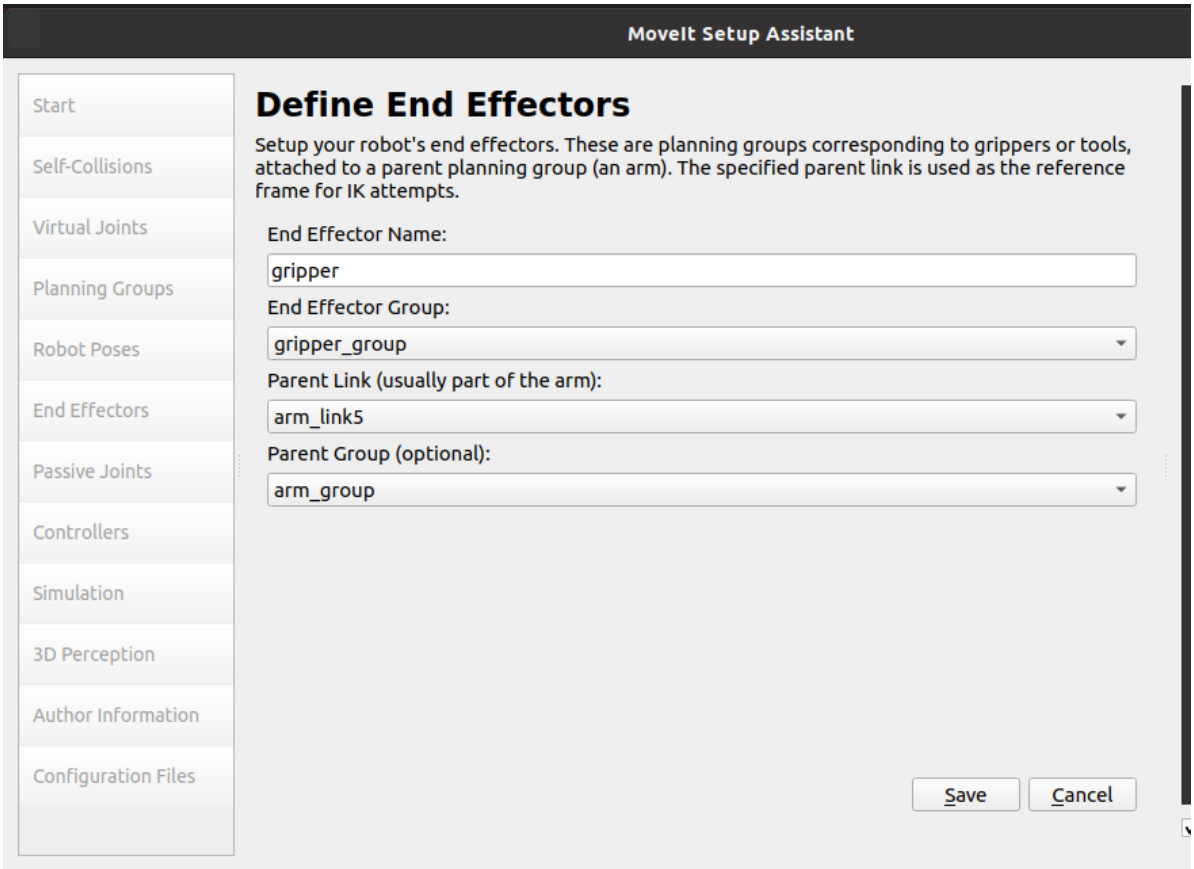


添加夹爪【open】姿态

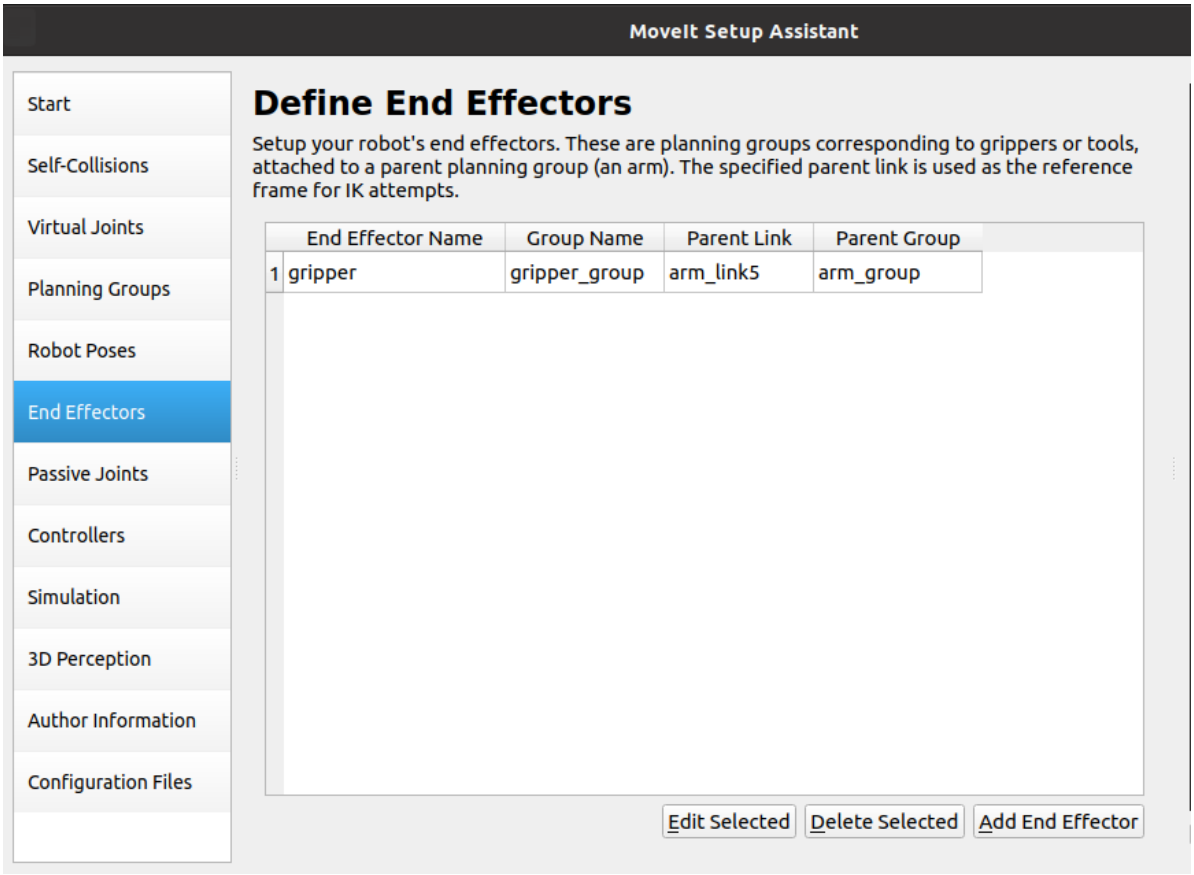


1.2.5、设置末端影响关节

选择【End Effectors】，如下图所示设置

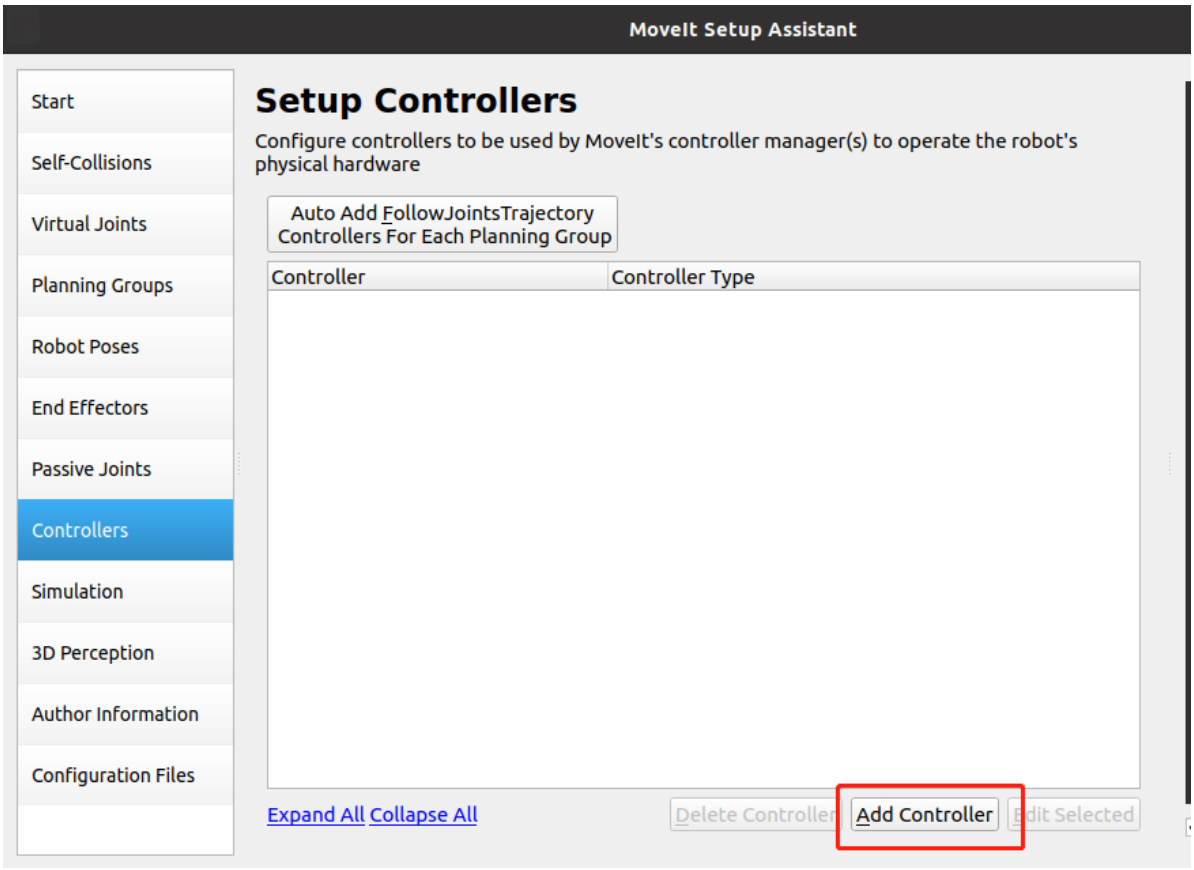


点击Save保存退出，设置完成如下图所示，

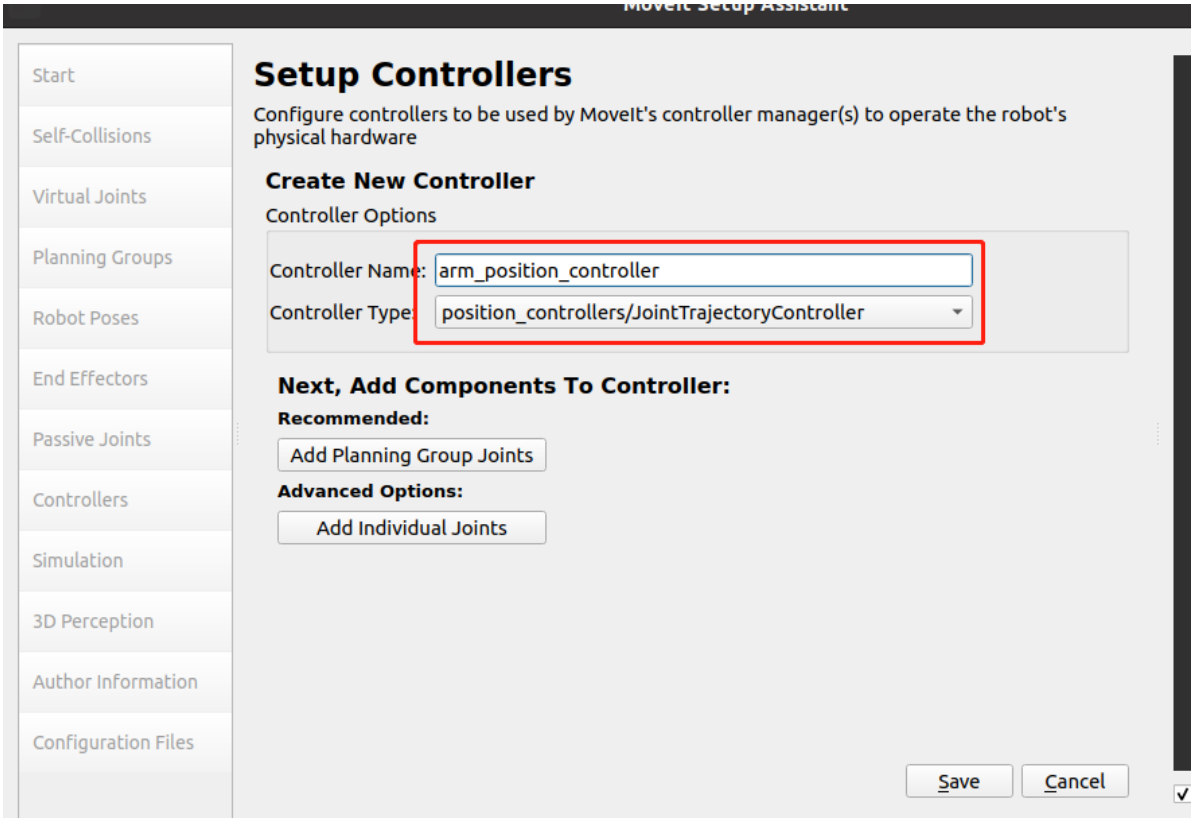


1.2.6、建立ROS控制器

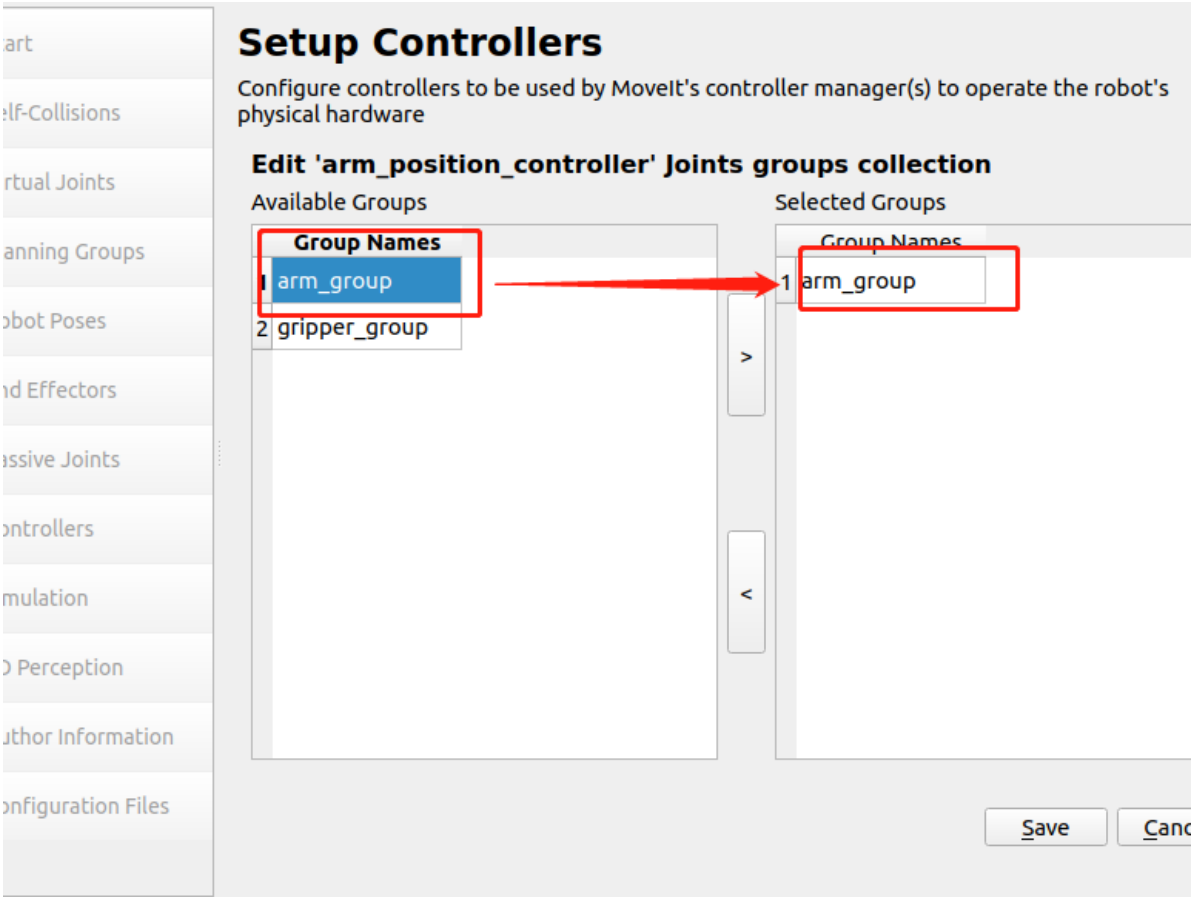
选择【Controllers】，点击Add Controller，



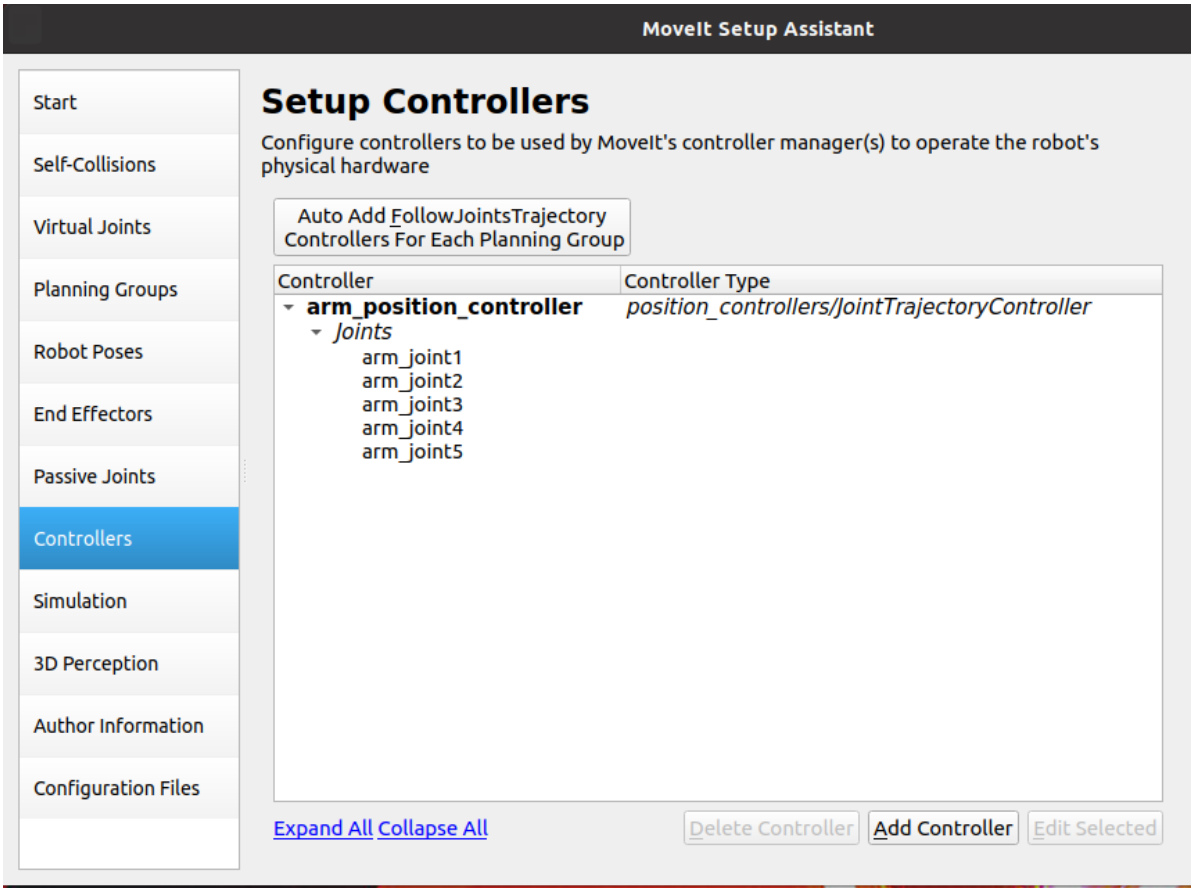
然后，Controller Name：arm_position_controller，Controller Type选择position_controllers/JointTrajectoryController



点击Add Planning Group Joints，选择arm_group，点击>，点击save保存。

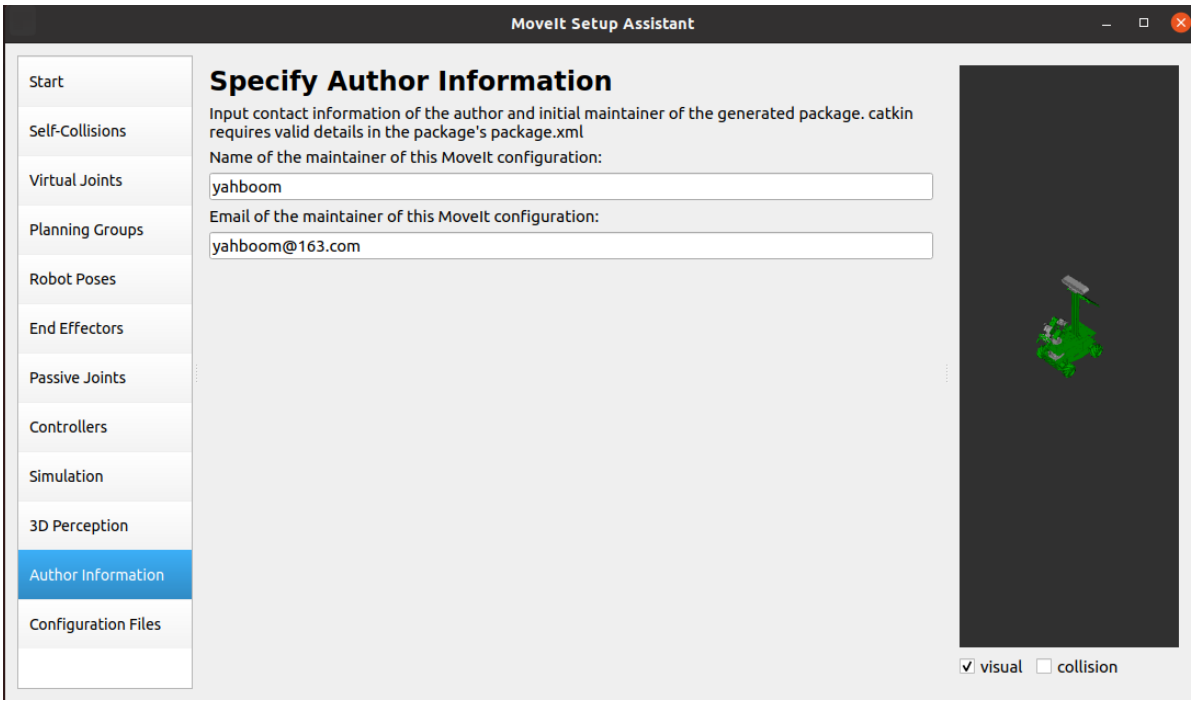


最后配置如下图所示，



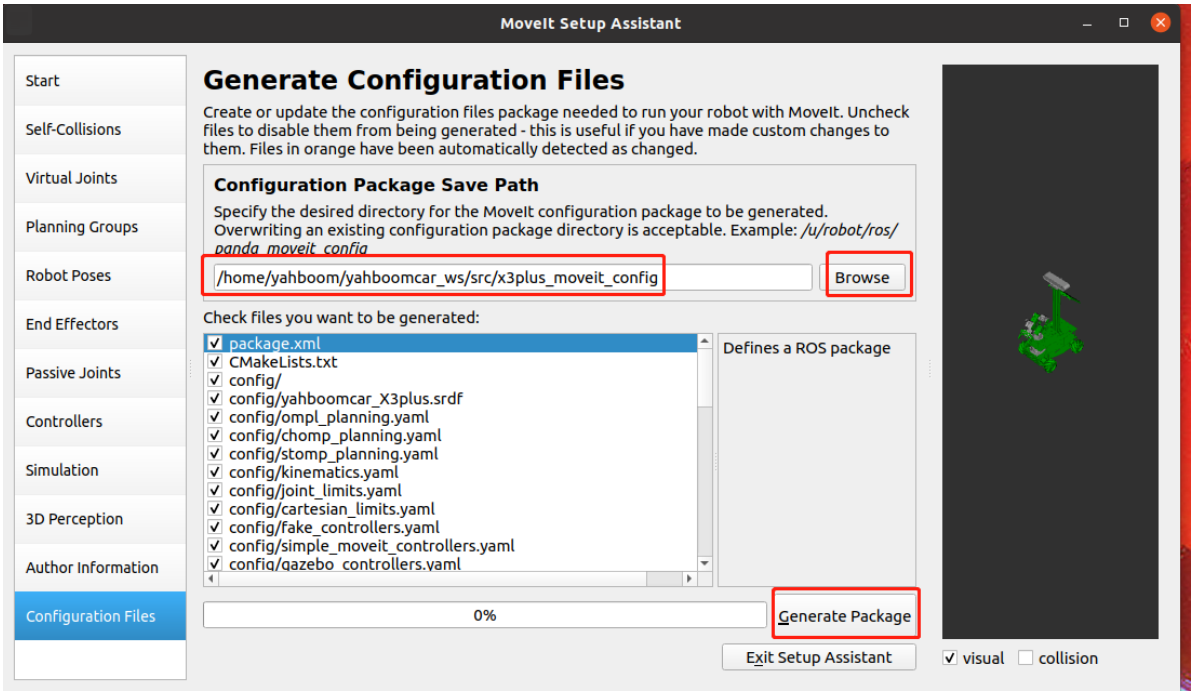
1.2.7、添加作者信息，不添加，则无法生成

选择【Author Information】，添加内容如下。

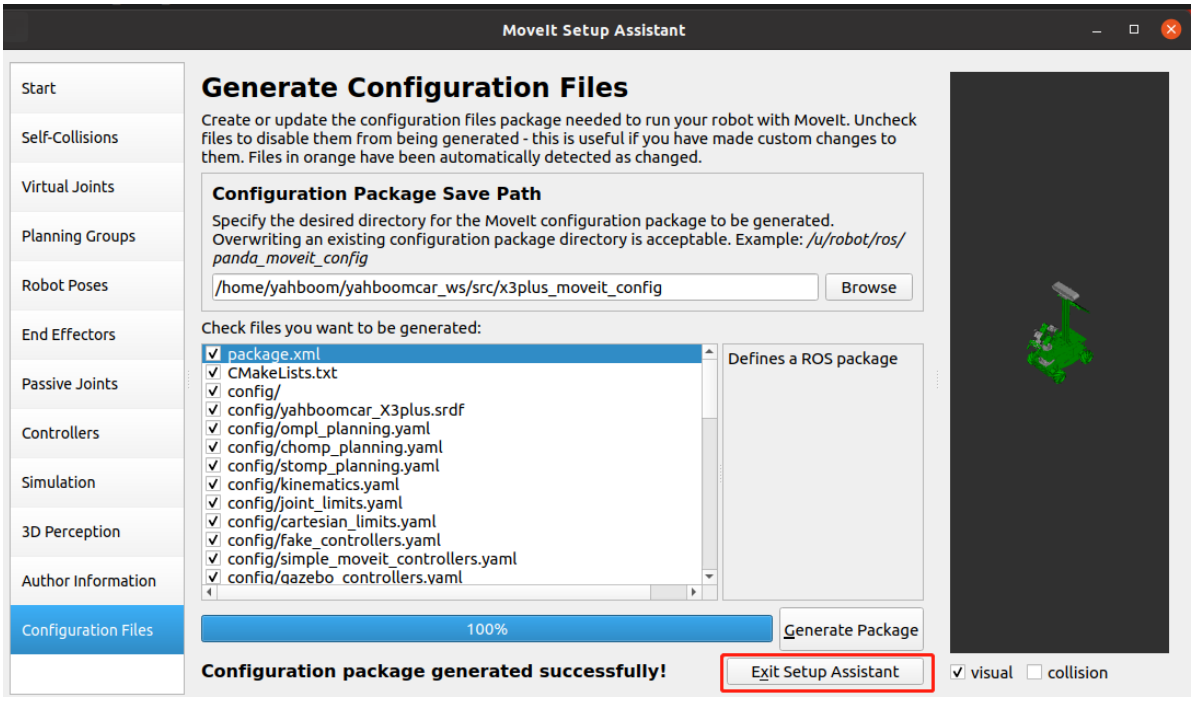


1.2.8、生成配置文件

选择【Configuration Files】，点击【Browse】，选择【x3plus_moveit_config】文件夹（该文件夹必须为空），点击【Generate Package】按钮，生成配置文件。完毕后，点击【Exit Setup Assistant】。



生成完毕后，点击Exit Setup Assistant退出配置。



1.3、MoveIt 配置包详解

打开刚刚创建好的【x3plus_moveit_config】文件夹，我们发现有config和launch两个文件夹。

1.3.1、config文件夹

- fake_controllers.yaml：这是虚拟控制器配置文件，方便我们在没有实体机器人，甚至没有任何模拟器（如gazebo）开启的情况下也能运行MoveIt。
- joint_limits.yaml：这里记录了机器人各个关节的位置速度加速度的极限，这些都会被用于以后的规划中。
- kinematics.yaml：运动规划组设置的东西，用于初始化运动学求解库
- yahboomcar_X3plus.srdf：这个是一个重要的MoveIt配置文件。
- ompl_planning.yaml：这里是配置OMPL各种算法的各种参数。

- SRDF文件：SRDF是moveit的配置文件，配合URDF使用。我们可以看到这是一个xml格式的配置文件，根是robot，并有一个属性name='yahboomcar_X3plus'。下面就是刚刚在Setup Assistant里面所设置的东西，包含了组群，位姿，终端控制器，虚拟关节，以及碰撞免测矩阵ACM的定义。理论上，只要有了srdf和urdf，我们就可以完全定义一个机器人moveit信息。

1.3.2、launch文件夹

- demo.launch：demo是运行的总结点，打开我们可以看到他include了其他的launch文件。
- move_group.launch：顾名思义，move group的功能是让一个规划组群动起来。默认的是使用ompl运动规划库。其他的都是设置一些基本参数，暂时可以略过。
- planning_context.launch：这里我们可以看到，定义了所使用的urdf和srdf文件，以及运动学求解库。不建议手动更改这些，但是如果你需要使用不同的urdf，srdf，可以在这里更改。
- setup_assistant.launch：如果你需要更改一些配置，那么可以直接运行。

1.4、配置验证

进入配置文件所在的工作空间，执行以下命令

```
cd ~/yahboomcar_ws                                          # 进入工作空间
catkin_make                                                 # 编译
source devel/setup.bash                                     # 更新系统环境
roslaunch arm_moveit_demo x3plus_moveit_demo.launch sim:=true # 启动moveIT
```

示例如下

